**Register Number:** 192425207

**Name:** Anand Paul. D

**CO1 – AT3**

**MLA0201 – Fundamentals of Machine Learning**

# Experiment 1: Probability Theory (20 Marks)

## Aim

To calculate the probability of each class (Malignant and Benign) in the Breast Cancer Wisconsin Diagnostic Dataset and predict the class of a new data instance based on the computed probabilities.

## Algorithm

1. Load the Breast Cancer Wisconsin Dataset.
2. Count the number of Malignant and Benign instances.
3. Calculate class probabilities using:
    - Probability = (Number of instances in class) / (Total instances)
4. Compare the probabilities.
5. Predict the class having the highest probability.

## Pseudocode

START

Load Breast Cancer Dataset

Count Malignant samples

Count Benign samples

Calculate P(Malignant)

Calculate P(Benign)

IF P(Benign) > P(Malignant)

Predict Benign

ELSE

Predict Malignant

END IF
Display probabilities and prediction

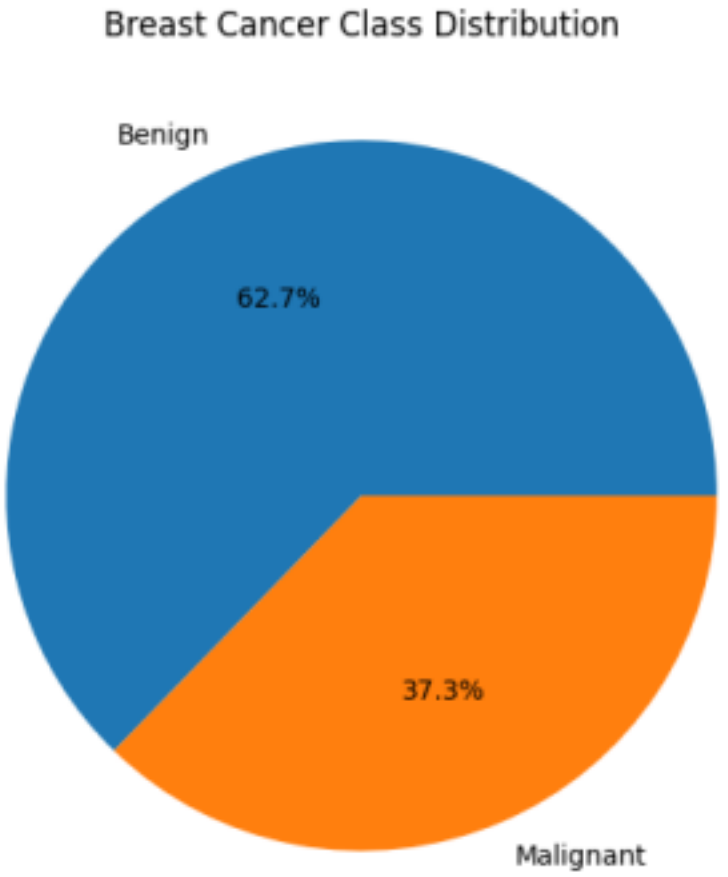
STOP

## Code:

```
# Experiment 1: Probability Theory
# Breast Cancer Wisconsin Diagnostic Dataset

from sklearn.datasets import load_breast_cancer
import pandas as pd

# Load dataset
data = load_breast_cancer()

# Create DataFrame
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Convert target values to class names
df['Class'] = df['target'].map({0: 'Malignant', 1: 'Benign'})

# Total number of samples
total_samples = len(df)

# Count instances of each class
malignant_count = len(df[df['Class'] == 'Malignant'])
benign_count = len(df[df['Class'] == 'Benign'])

# Calculate probabilities
p_malignant = malignant_count / total_samples
p_benign = benign_count / total_samples

print("Total Samples:", total_samples)
print("Malignant Cases:", malignant_count)
print("Benign Cases:", benign_count)

print("\nClass Probabilities:")
print("P(Malignant) =", round(p_malignant, 4))
print("P(Benign) =", round(p_benign, 4))

# Predict class of a new instance using prior probabilities
print("\n\nPrediction for a New Data Instance:")

if p_benign > p_malignant:
    predicted_class = "Benign"
else:
    predicted_class = "Malignant"

print("Predicted Class =", predicted_class)
```

## Output:

```
Total Samples: 569
Malignant Cases: 212
Benign Cases: 357

Class Probabilities:
P(Malignant) = 0.3726
P(Benign) = 0.6274

Prediction for a New Data Instance:
Predicted Class = Benign
```




**Result:** The probabilities of Malignant and Benign classes were calculated. Since P(Benign) > P(Malignant), the new instance was classified as Benign.

## Experiment 2: Bayes Decision Theory (15 Marks)

**Aim**

To implement Bayes Theorem using the SMS Spam Collection Dataset and classify a message as Spam or Ham.

**Algorithm**

1. Load SMS dataset.
2. Convert text messages into numerical vectors.
3. Train a Naive Bayes classifier.
4. Input a new SMS message.
5. Calculate posterior probabilities.
6. Assign the class with the highest posterior probability.

**Pseudocode**

START

Load SMS Dataset

Convert text to vectors

Train Naive Bayes Model

Input New Message

Calculate Posterior Probabilities

IF P(Spam|Message) > P(Ham|Message)

Classify as Spam

ELSE

Classify as Ham
END IF

Display Result

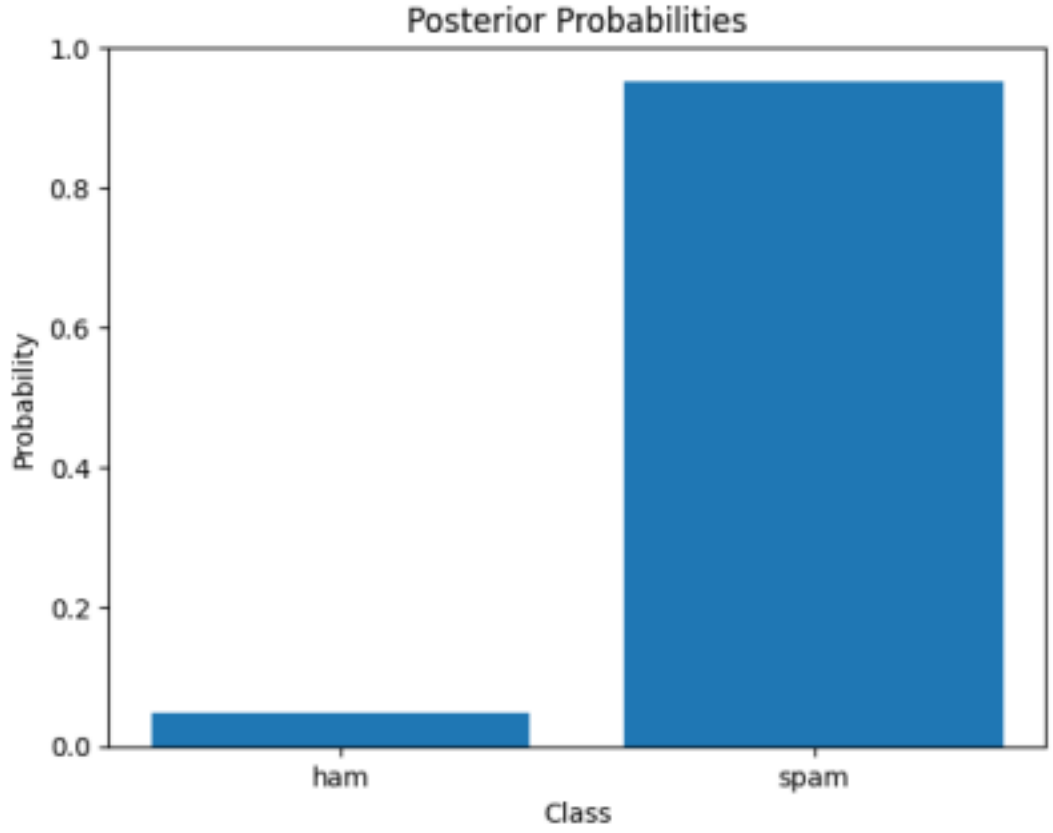
STOP

## Code:

```python
# Experiment 2: Bayes Decision Theory
# SMS Spam Collection Dataset

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Sample SMS Dataset
data = {
    'label': ['ham','spam','ham','spam','ham','spam','ham'],
    'message': [
        'How are you',
        'Win cash prize now',
        'Lets meet tomorrow',
        'Claim your free reward',
        'Call me later',
        'Congratulations you won lottery',
        'Good morning'
    ]
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

# Features and Target
X = df['message']
y = df['label']

# Convert text into numerical features
vectorizer = CountVectorizer()
X_vec = vectorizer.fit_transform(X)

# Train Naive Bayes Model
model = MultinomialNB()
model.fit(X_vec, y)

# New message for testing
new_message = ["Congratulations! You won a free cash prize"]

# Transform message
new_vec = vectorizer.transform(new_message)

# Prediction
```

## Output:

```
Dataset:
  label                          message
0   ham                      How are you
1  spam              Win cash prize now
2   ham              Lets meet tomorrow
3  spam          Claim your free reward
4   ham                    Call me later
5  spam  Congratulations you won lottery
6   ham                     Good morning

New Message:
Congratulations! You won a free cash prize

Posterior Probabilities:
ham: 0.0475
spam: 0.9525

Predicted Class: SPAM
```




**Result:** The posterior probabilities were calculated using Bayes theorem and the message was classified as Spam.

## Experiment 3: Information Theory (15 Marks)

**Aim**

To calculate Entropy and Information Gain for all attributes in the Play Tennis Dataset and identify the attribute with the highest Information Gain.

## Algorithm

1. Load Play Tennis Dataset.
2. Calculate dataset entropy.
3. For each attribute:
    - Split dataset based on attribute values.
    - Calculate weighted entropy.
    - Calculate Information Gain.
4. Compare Information Gain values.
5. Select attribute with highest Information Gain.

## Pseudocode

**START**

**Load Play Tennis Dataset**

**Calculate Dataset Entropy**

**FOR each Attribute**

**Calculate Entropy after split**

**Calculate Information Gain**

**END FOR**

**Find Maximum Information Gain**

**Display Best Attribute**

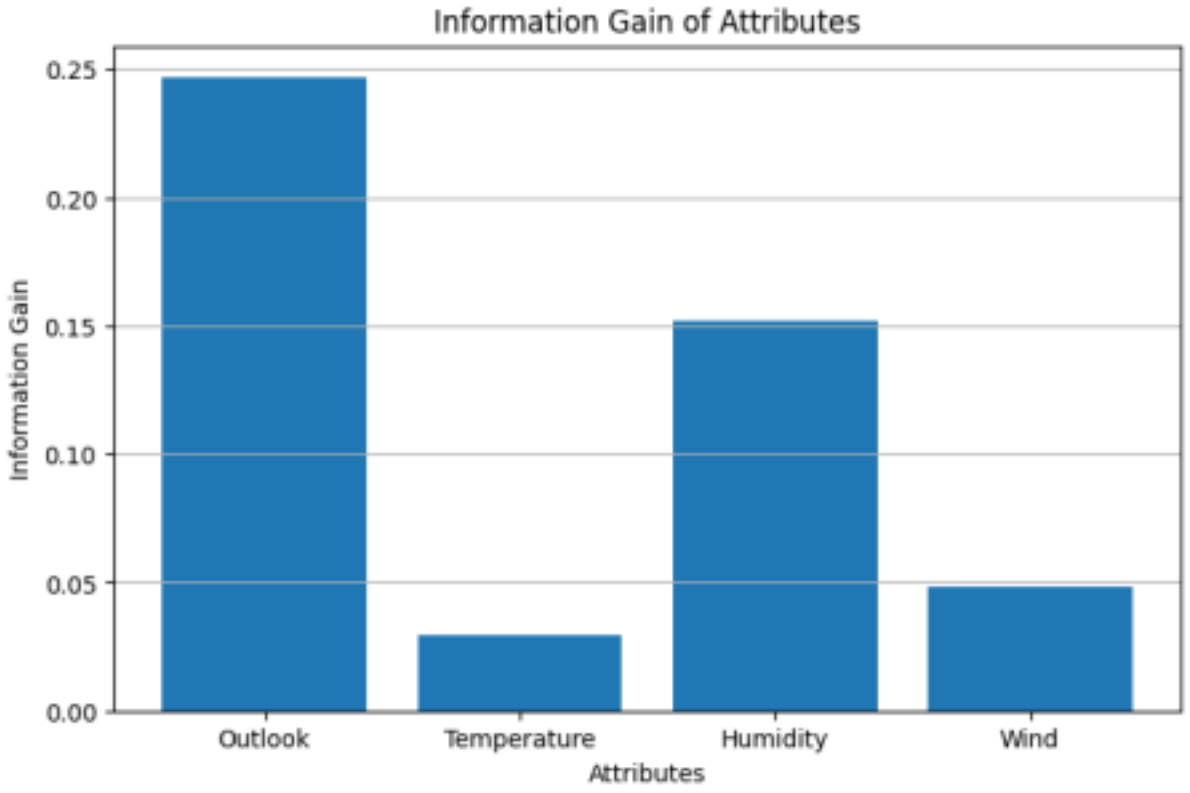
**STOP**

## Code:

```python
import pandas as pd
import numpy as np
from math import log2

# Play Tennis Dataset
data = {
    'Outlook':['Sunny','Sunny','Overcast','Rain','Rain','Rain',
               'Overcast','Sunny','Sunny','Rain','Sunny',
               'Overcast','Overcast','Rain'],
    'Temperature':['Hot','Hot','Hot','Mild','Cool','Cool',
                   'Cool','Mild','Cool','Mild','Mild',
                   'Mild','Hot','Mild'],
    'Humidity':['High','High','High','High','Normal','Normal',
                'Normal','High','Normal','Normal','Normal',
                'High','Normal','High'],
    'Wind':['Weak','Strong','Weak','Weak','Weak','Strong',
            'Strong','Weak','Weak','Weak','Strong',
            'Strong','Weak','Strong'],
    'PlayTennis':['No','No','Yes','Yes','Yes','No',
                  'Yes','No','Yes','Yes','Yes',
                  'Yes','Yes','No']
}

df = pd.DataFrame(data)

# Function to calculate entropy
def entropy(target):
    values, counts = np.unique(target, return_counts=True)
    ent = 0
    for count in counts:
        p = count / sum(counts)
        ent -= p * log2(p)
    return ent

# Function to calculate information gain
def information_gain(data, attribute, target='PlayTennis'):
    total_entropy = entropy(data[target])

    values, counts = np.unique(data[attribute], return_counts=True)

    weighted_entropy = 0
    for value, count in zip(values, counts):
        subset = data[data[attribute] == value]
        weighted_entropy += (count / len(data)) * entropy(subset[target])

    return total_entropy - weighted_entropy
```

## Output:

```
Entropy of Dataset = 0.9403

Information Gain of Attributes:
Outlook: 0.2467
Temperature: 0.0292
Humidity: 0.1518
Wind: 0.0481

Attribute with Highest Information Gain:
Outlook
```




**Result:** The entropy of the Play Tennis dataset was calculated as 0.9403. Information Gain was computed for all attributes. Outlook produced the highest Information Gain (0.2467) and was selected as the best attribute for splitting the dataset.